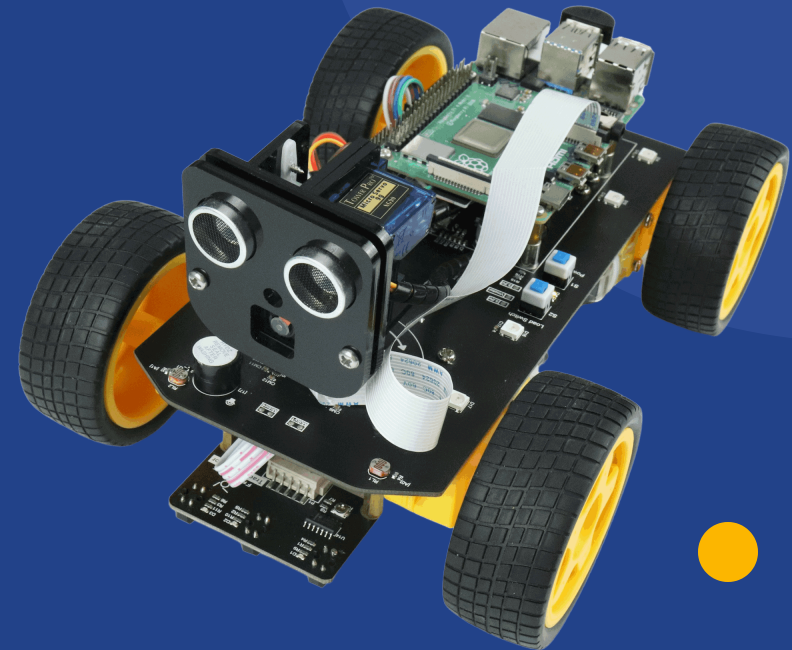




LESSON 5 – LINE FOLLOWER

Machine Learning with Raspberry Pi & Smart Car

Today the car follows a line on the floor – the classic robot challenge – using three infrared sensors underneath.



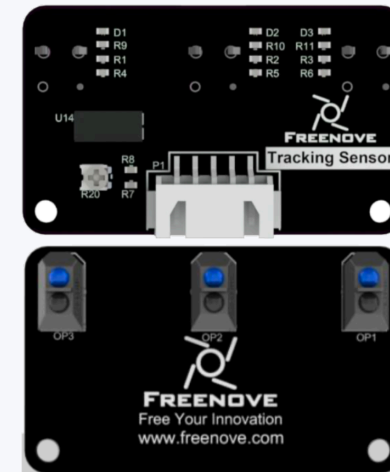


LESSON 5 – LINE FOLLOWER

Follow the line on the floor

Last time the car avoided obstacles and chased light. Today it tracks a line – with a new tool.

- Lesson 4: the car **dodged obstacles** and **chased light** on its own.
- **Today:** it follows a **line on the floor** – the classic robot challenge.
- New tool: three **infrared (IR) sensors** underneath the car.





LESSON 5 – LINE FOLLOWER

Today's goal: stay on the line

Build a line-following car that senses where the line is and steers to stay on it.

- 1 **Read** three IR sensors – Left, Middle, Right.
- 2 **Figure out** where the line is relative to the car.
- 3 **Steer** the motors to stay on the line.



New skill: read the Pi's pins directly with **RPi.GPIO**.



LESSON 5 – LINE FOLLOWER

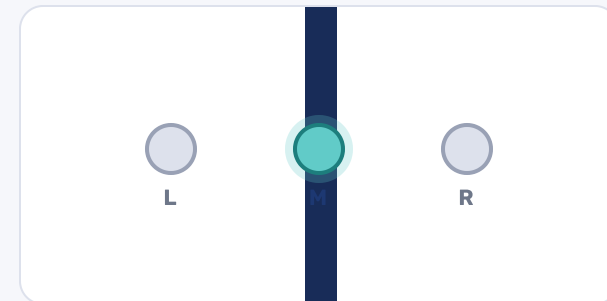
The three IR sensors

Three sensors under the car – Left, Middle, Right – each reads a simple yes/no.

1 it sees the dark **line**

0 it sees the light **floor**

Together, the three answers tell us **where the line is** relative to the car.



The **middle** sensor is over the line → the car is centered.

LESSON 5 – LINE FOLLOWER • THE CLEVER BIT

Three sensors → one number

Pack three yes/no answers into one number: `Left×4 + Middle×2 + Right×1`. Read it in binary as `0b L M R`.

Binary (L M R)	#	What the car does
<code>0b000</code>	0	nothing seen → stop (lost)
<code>0b001</code>	1	right only → turn right a bit
<code>0b010</code>	2	centered → go straight
<code>0b011</code>	3	middle + right → turn right harder
<code>0b100</code>	4	left only → turn left a bit
<code>0b101</code>	5	both edges → stop (rare)
<code>0b110</code>	6	left + middle → turn left harder
<code>0b111</code>	7	all on → stop (junction)

WHY IT'S EXACTLY 8

$$2 \times 2 \times 2 = 2^3 = 8$$

Each sensor is **1 bit** – 2 choices. Three sensors stack into **3 binary digits**, so counting `000` → `111` lists all **8 patterns**, numbered **0-7**.

Each digit is one sensor: `0b L M R`. One number per pattern keeps the steering code tiny.



LESSON 5 – LINE FOLLOWER • THE FILE, CHUNK 1

Start the file: imports

New import: `RPi.GPIO` lets us read the Pi's pins directly.

- `import car_setup` first – it finds the car's modules.
- `import RPi.GPIO as GPIO` – the `as` gives it a short nickname.
- We still drive with **Motor** from earlier lessons.

```
lesson_5_line_follower.py

"""Lesson 5: Line Follower."""

import car_setup      # add the car's code folders
import time

# read the Pi's pins directly
import RPi.GPIO as GPIO

from motor import Motor    # driving wheels
```



LESSON 5 – LINE FOLLOWER • THE FILE, CHUNK 2

Settings: sensor pins

Each sensor is plugged into a Pi pin (BCM numbering). Naming them keeps the code readable.

- **BCM** is one of the Pi's pin-numbering schemes.
- You don't need the theory – just that these match the **wiring**.
- Left → 14 , Middle → 15 , Right → 23 .

```
lesson_5_line_follower.py

# ---- sensor pins (BCM numbering) ----
IR_LEFT = 14
IR_MIDDLE = 15
IR_RIGHT = 23
```



LESSON 5 – LINE FOLLOWER • THE FILE, CHUNK 3

Settings: driving speeds

Each move is four wheel speeds (LF, LB, RF, RB) – same as Lesson 2.

- **Soft** turns nudge the car a little.
- **Hard** turns swing harder when it's more off-line.
- All four numbers run from `-4095` to `4095`.

```
lesson_5_line_follower.py

# ---- driving speeds (LF, LB, RF, RB) ----
FORWARD = (800, 800, 800, 800)
SOFT_LEFT = (-1500, -1500, 2500, 2500)
HARD_LEFT = (-2000, -2000, 4000, 4000)
SOFT_RIGHT = (2500, 2500, -1500, -1500)
HARD_RIGHT = (4000, 4000, -2000, -2000)
STOP = (0, 0, 0, 0)
```



LESSON 5 – LINE FOLLOWER • THE FILE, CHUNK 4

setup(): the pins are inputs

Create the motor, then tell the Pi the IR pins are **inputs** – we READ them.

- `GPIO.BCM` picks the pin-numbering scheme.
- `GPIO.IN` = data flows from **sensor** → **Pi**.
- `setwarnings(False)` hides harmless warnings.

```
lesson_5_line_follower.py

motor = None

def setup():
    """Make the motor; set sensor pins as inputs."""
    global motor
    motor = Motor()
    GPIO.setwarnings(False)    # hide harmless warnings
    GPIO.setmode(GPIO.BCM)    # BCM pin numbering
    GPIO.setup(IR_LEFT, GPIO.IN)    # IN = we READ these
    GPIO.setup(IR_MIDDLE, GPIO.IN)
    GPIO.setup(IR_RIGHT, GPIO.IN)
```



LESSON 5 – LINE FOLLOWER • THE FILE, CHUNK 5

read_sensors(): combine the three

Read each pin (1 or 0), then pack them into one number – exactly the binary trick.

- Each `GPIO.input` reads one sensor: `1` = line, `0` = floor.
- `Left×4 + Middle×2 + Right×1` builds the one number.
- That number is the binary `0b L M R` from slide 5.

```
lesson_5_line_follower.py

def read_sensors():
    """One number for the line's position."""
    left = GPIO.input(IR_LEFT)      # 1 = line
    middle = GPIO.input(IR_MIDDLE)
    right = GPIO.input(IR_RIGHT)
    return left * 4 + middle * 2 + right
```



LESSON 5 – LINE FOLLOWER • THE FILE, CHUNK 6

choose_move(): one move per pattern

Match the sensor number to a steering move. The `0b...` values read as *L M R*.

- Read each `0b` value out loud as **L-M-R**.
- Anything else – all three, or none –
→ **stop**.

```
lesson_5_line_follower.py

def choose_move(sensors):
    """Pick wheel speeds for this pattern."""
    if sensors == 0b010:    # middle only -> straight
        return FORWARD
    if sensors == 0b100:    # left only -> left a bit
        return SOFT_LEFT
    if sensors == 0b110:    # left + middle -> harder
        return HARD_LEFT
    if sensors == 0b001:    # right only -> right a bit
        return SOFT_RIGHT
    if sensors == 0b011:    # middle + right -> harder
        return HARD_RIGHT
    return STOP             # all three, or none -> stop
```



LESSON 5 – LINE FOLLOWER • THE FILE, CHUNK 7

loop(): sense and steer, fast

Read → decide → drive, over and over – the same Sense → Think → Act loop, just faster.

- **Read** the sensors → **decide** the move → **drive** the motors.
- The short `sleep(0.02)` (20 ms) keeps steering smooth.
- Same loop as Lesson 4 – speed matters for smooth lines.

```
lesson_5_line_follower.py

def loop():
    """Read the sensors and drive, over and over."""
    while True:
        sensors = read_sensors()      # where is the line?
        move = choose_move(sensors)   # how to drive?
        motor.setMotorModel(move[0], move[1], move[2], move[3])
        time.sleep(0.02)              # keeps steering smooth
```



LESSON 5 – LINE FOLLOWER • THE FILE, CHUNK 8

Clean up: destroy()

Stop the wheels, then **release the pins** with `GPIO.cleanup()`.

- `setMotorModel(0, 0, 0, 0)` stops the wheels first.
- `GPIO.cleanup()` hands the pins back – it's new this lesson.
- Releasing pins lets other programs use them next time.

```
lesson_5_line_follower.py

def destroy():
    """Stop the wheels and release the pins."""
    if motor is not None:
        motor.setMotorModel(0, 0, 0, 0)
    GPIO.cleanup()          # hand the pins back
```



LESSON 5 – LINE FOLLOWER • THE FILE, CHUNK 9

The main block

The familiar safe wrapper: `try` runs the work, `finally` always cleans up.

- `try` runs `setup()` then `loop()`.
- `except KeyboardInterrupt` catches **Ctrl+C**.
- `finally` guarantees `destroy()` runs no matter what.
- Fifth time seeing this block – you should predict it now.

lesson_5_line_follower.py

```
if __name__ == "__main__":
    try:
        setup()
        loop()
    except KeyboardInterrupt:
        pass
    finally:
        destroy()
```



LESSON 5 – LINE FOLLOWER

You built the whole file

Complete `lesson_5_line_follower.py` – and `choose_move()` stays tiny thanks to the binary trick.



The **teal** read and **amber** decide steps are the heart – three sensors in, one move out.



Three sensors, one number, a car that follows a line.



LESSON 5 – LINE FOLLOWER

Run it

Put the car on a **line track** – black tape on a light floor – then run on the **Pi**.

- A simple **oval / loop** of tape works best.
- The car should drive along the line on its own.
- Press `Ctrl+C` to stop. Use `sudo` for the run command.

 Pi terminal

```
cd Smart-Car-Level-3/Code/User  
sudo python3 lesson_5_line_follower.py
```



Make the line dark and the floor light – good contrast helps the sensors.



LESSON 5 – LINE FOLLOWER

Try it yourself

Tune the speeds, watch the sensor numbers, then push the track further.

- Speed up `FORWARD` for a faster lap – does it still hold the line?
- Make `HARD_LEFT` / `HARD_RIGHT` stronger for sharp corners.
- **Debug move:** `print(sensors)` in `loop()` and move the car by hand.

CHALLENGE

Build a tougher track

Make a **sharper** line track, then tune the speeds so the car can handle the tighter corners.



LESSON 5 – LINE FOLLOWER

Common issues

If the following looks off, it's almost always one of these.



Ignores the line

Check the IR sensors face the floor; is the line dark enough?



Steers the wrong way

Left/Right sensors may be swapped – check pins `14 / 15 / 23`.



Stops constantly

Reading "all" or "none" – adjust sensor height or track contrast.



"Pin already in use"

Harmless – `GPIO.setwarnings(False)` hides it.



Check this first

Sensor height above the floor matters a lot
– a few millimeters can fix erratic reads.



LESSON 5 – LINE FOLLOWER

Halfway there – the car follows a line!

Five lessons down. You read the Pi's pins and turned three sensors into one decision.

- ✓ Read the Pi's pins directly with **RPi.GPIO**
- ✓ Used a **binary trick** to turn three sensors into one number
- ✓ 🎉 You're **halfway through the course** – great momentum!

UP NEXT

Lesson 6 Camera stream

Give the car a **camera** and stream live video over Wi-Fi – the course shifts from driving to *seeing*.



• COACH ONLY • HIDDEN SLIDE

Halfway milestone – parent update

Don't present this slide. You're at the course midpoint – time for a personal progress note home.

- 1 Send each parent / guardian a progress update**
You're halfway through the course – reach out to every student's family individually.

- 2 Note what's going well**
Call out a few **good things** about their child's work and engagement so far.

- 3 Note what they could work on**
Add anything to improve (if anything). **Make it personal** – not a copy-paste.

REMEMBER

Families value a **warm, specific** note far more than a generic one. A sentence or two per student is plenty.



• COACH ONLY • HIDDEN SLIDE

End-of-class checklist

Don't present this slide. Run through it right after class.

1

Post the after-class recap

Summarise what you covered on the class page so students can catch up.

2

Log absent students

Post anyone who missed class in the **#attendance** Discord channel.

3

Prep for next class

Charge the cars and have a **camera + Pi** ready for **Lesson 6 – camera stream**.

EVERY CLASS

✓ Post a class announcement

✓ Introductions if needed

✓ Note absences in #attendance